

Client-сервер — язык и база данных

Дата: 2026-08-20

1. Решение

	Выбор	Одной строкой почему
Язык сервера	Go	Один статический бинарник на все платформы с одной машины сборки ; низкий порог входа; жанровый стандарт self-hosted (PocketBase, ntfy)
Клиентское ядро	Rust	iOS-расширение уведомлений требует нативного ядра, вызываемого из Swift; Go-рантайм с GC внутри расширения не возит никто из мессенджеров
База данных	SQLite через <code>modernc.org/sqlite</code> (чистый Go, без CGO)	Масштаб «круг из ~10 человек»; ноль администрирования; бэкап — один файл; путь PocketBase
TLS	<code>crypto/tls</code> (стандартная библиотека Go)	Без зависимости от системного OpenSSL — ничего не требуется на машине пользователя
Поставка	Один самодостаточный исполняемый файл на платформу (<code>CGO_ENABLED=0</code>)	«Скачал и запустил» — без рантаймов, интерпретаторов и внешних библиотек
Платформы	Windows x64 · macOS arm64/x64 · Linux x64/arm64 · Raspberry Pi (arm64/armv7)	Все цели собираются с одного Linux-раннера

Принятая цена: протокол «приложение ↔ client-сервер» существует в двух реализациях — Rust-ядро на клиентской стороне и Go на серверной, — потому что делить ядро они не могут в любом случае. Масштаб дублирования зависит от Q1: при E2EE, заканчивающемся на устройстве, сервер работает с непрозрачными блобами, и общая часть сжимается до кодека конверта и проверки подписи Ed25519. **Отсюда требование: Q1 решается до первой строчки протокола.**

Критерии выбора: деплой на все десктопные платформы и ARM; размер дистрибутива; простота установки для нетехнического пользователя; лёгкая и стабильная БД; восстановимость; порог входа и стоимость сопровождения. Сравнение кандидатов (Python / Rust / Go / Dart / Elixir) — §2.

2. Кандидаты

Легенда: ● хорошо · ● с оговорками · ● дисквалифицирует

	Все платформы + Pi	Размер	«Один файл и запустил»	SQLite внутри	Одно ядро с приложением
Rust	●	● единицы МБ	●	●	●
Go	●	● ~12 МБ	●	●	●
Dart	●	● ~4–5 МБ	●	●	●
Elixir	●	● 23–33 МБ + распаковка	●	●	●
Python	●	● десятки МБ	●	●	●

Последняя колонка — про клиентское ядро, не про сервер. Клиентская часть протокола и криптоядро **обязаны** быть нативными: на iOS содержимое уведомлений забирает Notification Service Extension — отдельный процесс, где Flutter/Dart не работают ([notification-delivery.md](#) §5). Ядро — Rust по модели libsignal (официальные биндинги Swift, Kotlin, TypeScript). Сервер на том же Rust дал бы единственную реализацию протокола — это отклонено в пользу скорости разработки и найма: асинх-сетевой сервер — самая крутая часть кривой обучения Rust, а дублирование при E2EE-на-устройстве сжимается до кодека конверта (см. §1).

Python отпадает на доставке: нужен либо интерпретатор на машине пользователя, либо PyInstaller-бандл на десятки мегабайт с хрупким запуском. Для нетехнического пользователя оба варианта — не «скачал и запустил».

Dart отпадает на поставке, и владение языком это не компенсирует: кросс-компиляция `dart compile exe` работает **только в Linux-цели** (с Dart 3.8/3.9; Windows и macOS собираются только нативно на своих ОС), Linux-бинарник привязан к glibc (musl официально не поддерживается), а однофайловость и автоматический бандл SQLite взаимоисключены — `package:sqlite3` подтягивает нативную библиотеку через hooks, но hooks не работают с `dart compile exe`, а `dart build cli` выдаёт каталог с `.so / .dll` рядом с `exe`. Серверная экосистема тонкая: shelf зрелый, но минимальный; Serverpod требует PostgreSQL; dart_frog — волонтёрский проект после ухода мейнтейнеров.

Elixir — язык с самой сильной моделью конкурентности и живучести из всех кандидатов (BEAM: миллионы легковесных процессов, деревья супервизии с автоперезапуском упавшего — прецедент Discord: миллионы одновременных подключений, причём с Rust-ядром через Rustler). Но для нашего продукта он спотыкается ровно о поставку. `mix release` производит **каталог** с вшитой

виртуальной машиной, привязанный к ОС/архитектуре машины сборки — кросс-сборки первого класса нет. Единственный живой путь к одному файлу — сторонний Burrito: **самораспаковывающийся** бинарник 23–33 МБ (замерено по реальным релизам), который при первом запуске разворачивает себя в AppData/Application Support; его собственный README называет подход экспериментальным, сборка на Windows-машине не поддерживается, 32-битный Raspberry Pi не покрыт. Windows-продакшен — служба через `erlsrv` от администратора без автостарта. SQLite и общее Rust-ядро при этом закрыты полноценно (precompiled NIF на 15 таргетов; Rustler зрелый). Показательно: **ни одного PocketBase-подобного однофайлового self-hosted продукта на BEAM не существует** — ejabberd ставится классической серверной установкой. Суперсилы Elixir — масштаб и кластерная живучесть — рассчитаны на нагрузку компании, а не круга из ~10 человек; его настоящая ниша — relay-сервер, если тот дорастёт до миллионов соединений.

Go — выбор. Кросс-компиляция всех целей с одной машины штатна (PocketBase собирает все 9 платформенных бинарников на одном Linux-раннере); порог входа минимален по замыслу языка (простота — задокументированная цель дизайна Go); разработчиков в ~6 раз больше, чем у ближайшей альтернативы по конкурентности (Stack Overflow 2025: Go 16,4%, Elixir 2,7%); весь жанр self-hosted-серверов для частных лиц — PocketBase, ntfy, Syncthing, Gitea, Caddy — написан на нём. Бинарник ~12 МБ — приемлемо. Как язык клиентского ядра Go не подходит (рантайм с GC в iOS-расширении), поэтому ядро остаётся Rust — цена в виде двух реализаций протокола принята в §1.

Rust — сильный второй для сервера и безальтернативный для ядра. Сервер на Rust дал бы единственную реализацию протокола, самые маленькие бинарники и отсутствие гонок данных на уровне компилятора — но пишется медленнее, и async-сервер требует самой дорогой части экспертизы Rust. Пересмотреть в пользу Rust-сервера, если Q1 решится в вариант «E2EE заканчивается на client-сервере» — тогда сервер участвует в криптографии и единая реализация ядра снова перевешивает.

3. Почему Go — подробнее

- **Поставка.** `CGO_ENABLED=0` даёт полностью статический бинарник; все цели — Windows, macOS, Linux, ARM (включая обе архитектуры Raspberry Pi) — собираются с одного Linux-раннера одной командой. Доказано продакшном PocketBase.
- **SQLite без CGO.** `modernc.org/sqlite` (чистый Go) сохраняет и однофайловость, и кросс-сборку; PocketBase ездит на нём в проде. Нет нативного кода — нет и класса отказов «сбой в NIF/FFI уронил процесс».

- **Порог входа и найм.** Простота — задокументированная цель дизайна Go; для команды, приходящей из Dart, это смена диалекта, а не парадигмы. Разработчиков в разы больше, чем у альтернатив (SO 2025: 16,4% против 2,7% у Elixir), найм дешевле.
- **Жанровый стандарт.** Весь self-hosted мир для частных лиц написан на Go — примеры ровно нашей задачи на каждый вопрос реализации.
- **Живучесть 24/7** обеспечивается службами ОС (systemd / launchd / служба Windows), которые обязательны в любом случае ради автозапуска — они же перезапускают упавший процесс.

⚠ Цена №1: **две реализации протокола** (Rust-ядро клиента + Go-сервер) — принята, масштаб зависит от Q1 (§1). Цена №2: GC — на масштабе десятка соединений не проявляется.

4. База данных: SQLite

Масштаб задачи: один пользователь или малый круг, один процесс, единицы одновременных клиентов. Отдельный СУБД-сервер здесь не решает ни одной задачи, но добавляет процесс, который нужно ставить, запускать, обновлять и чинить.

	Выбор / отказ	Почему
SQLite	🟢 выбор	Через modernc.org/sqlite — порт SQLite на чистый Go, вкомпилирован в бинарник: ни системных зависимостей, ни отдельного процесса, ни CGO. Актуален (соответствует свежей SQLite), в проде у RocketBase. Оговорки: одно соединение — одна горутина (у SQLite и так один писатель); версия modernc.org/libc пинится в go.mod
PostgreSQL	🔴	Требует установки, службы, обновлений и роли администратора — которой в модели NOX не существует . Оправдан на масштабе компании, не пользователя
Embedded KV (RocksDB, sled, redb)	🔴	Нет SQL и штатных инструментов осмотра; RocksDB заметно раздувает бинарник (Matrix-серверы на нём — 60–105 МБ)
Плоские файлы	🔴	Нет транзакций и запросов; при отключении питания целостность — наша проблема, а не проверенного движка

Режим: WAL + `synchronous=FULL`. Дешёвое железо умеет врать про fsync — гарантии любой БД на нём аннулируются, поэтому источник истины — устройство, а сервер хранит то, что можно вывести заново (границы хранения — Q1).

Восстановимость — свойство выбора, а не отдельная подсистема. Вся база — один файл: бэкап — копия файла (`VACUUM INTO` для консистентного снимка на живом сервере), восстановление — положить файл рядом с бинарником и запустить. Худший случай — потеря узла целиком — по архитектуре развёртывания и так «переразвернуть и спариться заново», а не катастрофа.

5. Прецеденты

Проект	Язык сервера	Хранилище	Масштаб
PocketBase	Go	SQLite, встроена («backend in 1 file»)	self-hosted
ntfy	Go	SQLite	self-hosted
Conduit / Tuvunel (Matrix)	Rust	встроенная RocksDB/SQLite, один бинарник, работает на Pi	self-hosted
Snikket	Lua (поверх Prosody)	плоские файлы (SQLite — опция)	self-hosted
chatmail (Delta Chat)	Postfix/Dovecot + Python-обвязка	почтовые хранилища Dovecot	self-hosted
SimpleX (SMP)	Haskell	память + журнал	self-hosted
libsignal	Rust (ядро, биндинги Swift/Kotlin/TS)	—	ядро клиента
Signal-Server	Java	DynamoDB + Redis + FoundationDB + S3	компания
Synapse (Matrix)	Python/Rust	PostgreSQL обязателен	компания

Закономерность жёсткая: **всё, что хостят у себя частные лица, — компилируемый язык, один бинарник, встроенное хранилище.** Внешние СУБД и JVM появляются только на масштабе компании. NOX-класс задачи — верхняя половина таблицы.

6. Открытые вопросы

	Вопрос	На ком	Реестр
●	Границы хранения: полный архив, спул с TTL или без состояния — на выбор БД не влияет, влияет на схему	владелец	Q1
●	Криптопримитивы Rust-ядра (клиентская сторона) — решается вместе с протоколом	архитектура	—
●	Входит ли запасной Android-телефон в площадки размещения. Go проходит (с-shared <code>.so</code> внутри APK) — язык на этом вопросе не висит	владелец	Q10